Software Requirements Specification (SRS)

For

AssignHub – Centralized Lab Assignment Submission and Evaluation System

Version: 3.0

Prepared by

Group Name: AssignHub Team

Vimal Solanki	24CEUBS148	24ceubs148@ddu.ac.in
Hanuj Trivedi	24CEUOZ158	24ceuoz158@ddu.ac.in

Instructor: *AMS*

Course: *SEPP*

Lab Section: *B4*

Date: *10th Feb, 2026*

Content

Content ii
Revision iii
1. Introduction 1
1.1 Document Purpose 1
1.2 Product Scope 1
1.3 Intended Audience and Document Overview 1
1.4 Definitions, Acronyms and Abbreviations 2
1.5 Document Conventions 2
1.6 References and Acknowledgements 2
2. Overall Description 3
2.1 Product Overview 3
2.2 Product Functionality 3
2.3 Design and Implementation Constraints 3
2.4 Assumption and Dependencies 4
3. Specific Requirements 5
3.1 External Interface Requirements 5
3.1.1 User Interfaces 5
3.1.2 Hardware Interfaces 5
3.1.3 Software Interfaces 5
3.2 Functional Requirements 6
3.3 Use Case Model 11
4. Other Non-functional Requirement 16
4.1 Performance Requirements 16
4.2 Safety and Security Requirements 16
4.3 Software Quality Attributes 17
5. Other Requirements 18
Appendix A – Data Dictionary 19
Appendix B – Group Log 20

Revision

Version	Primary Author(s)	Description of Version	Date Completed
Draft 1.0	Solanki Vimal & Hanuj Trivedi	Initial draft of AssignHub SRS completed, covering project overview, system purpose and scope, user characteristics, operating environment, design constraints, assumptions, and core functional requirements	*12/01/2026*
Draft 2.0	Solanki Vimal & Hanuj Trivedi	Added use case model and detailed use case descriptions for core system functionalities.	*23/01/2026*
Draft 3.0	Solanki Vimal & Hanuj Trivedi	Added detailed non-functional requirements and other system requirements. Completed Appendices including Data Dictionary and Group Log, and performed final review to produce the complete and finalized Software Requirements Specification.	*09/02/2026*

1. Introduction

AssignHub is a centralized web-based Lab Assignment Submission and Evaluation System designed to streamline and automate the management of academic lab assignments in educational institutions. The system addresses challenges associated with manual and semi-digital assignment handling, such as missed deadlines, misplaced submissions, lack of transparency, and delayed evaluation. AssignHub enables students to submit assignments online, faculty to evaluate and provide feedback efficiently, and administrators to manage academic structures and users through a single integrated platform.

This section provides an overview of the AssignHub project and outlines the purpose, scope, and structure of this Software Requirements Specification (SRS). It also introduces the intended audience, key terminology, document conventions, and references required to understand the system requirements.

1.1 Document Purpose

The purpose of this Software Requirements Specification (SRS) document is to define the requirements for **AssignHub – Centralized Lab Assignment Submission and Evaluation System (Version 1.0)**. This document serves as a formal specification of the system's functional and non-functional requirements and establishes a common understanding among stakeholders regarding what the system is expected to accomplish.

This SRS covers the complete AssignHub system, including user authentication, assignment creation and submission, evaluation and feedback management, academic structure administration, notifications, and reporting. It is intended to act as a reference for system design, development, testing, and evaluation, without specifying implementation details.

1.2 Product Scope

AssignHub is a web-based academic management system intended for use in educational institutions such as universities and colleges. The system enables faculty members to create and manage lab assignments, students to submit assignments electronically and track their status, and administrators to manage users, courses, and academic data.

The primary goals of ***AssignHub*** are to eliminate manual assignment handling, improve transparency in submission and evaluation, reduce administrative overhead, and ensure timely feedback to students. By providing centralized storage, role-based access, and real-time status updates, the system enhances academic efficiency and supports structured decision-making for faculty and administrators.

1.3 Intended Audience and Document Overview

This document is intended for project evaluators and instructors, developers, testers, and academic administrators who wish to understand the AssignHub system. Each reader may refer to the sections most relevant to their role.

The document is organized into multiple sections: Section 1 introduces the project and defines its scope and purpose, Section 2 provides an overall description of the system, Section 3 specifies detailed functional requirements and use cases, and Section 4 describes non-functional requirements. Additional supporting information is provided in the appendices. Readers are advised to begin with the introductory sections before reviewing the detailed requirements.

1.4 Definitions, Acronyms and Abbreviations

Term	Description
Admin	*User with administrative privileges to manage users, courses, and system settings*
Assignment	*A lab task created by faculty for student submission*
DBMS	*Database Management System*
Faculty	*Instructor responsible for creating and evaluating assignments*
SRS	*Software Requirements Specification*
UI	*User Interface*
User	*Any authenticated system user (Student, Faculty, or Admin)*

1.5 Document Conventions

This document follows the IEEE Software Requirements Specification (SRS) standard. The text is written using Arial font with size 11 or 12, single-spaced, and formatted with 1-inch margins on all sides. Section and subsection headings use a numbered hierarchical structure for clarity and consistency.

Functional requirements are labeled using a structured format (R.x.y) to ensure traceability throughout the document. Tables and bullet points are used where appropriate to improve readability and organization of information.

1.6 References and Acknowledgements

- IEEE Std 830-1998 – IEEE Recommended Practice for Software Requirements Specifications: https://ieeexplore.ieee.org/document/720574
- Course materials and project guidelines provided by the institution

The authors acknowledge the guidance and support provided by the course instructor and the reference materials used during the development of the AssignHub system.

2. Overall Description

2.1 Product Overview

AssignHub is a new, self-contained web-based Lab Assignment Submission and Evaluation System designed to replace traditional manual and semi-digital methods of managing academic lab work in educational institutions. The system provides a centralized platform for assignment creation, submission, evaluation, and academic data management, thereby improving efficiency, transparency, and reliability in academic workflows.

AssignHub operates within an institutional academic environment and interacts with students, faculty members, and administrators. Students use the system to view assignments, submit lab work, and track evaluation status. Faculty members access the system to create assignments, monitor submissions, evaluate work, and provide feedback. Administrators manage users, courses, batches, and system configurations. All assignment, submission, evaluation, and user data are maintained in a centralized database to ensure consistency and secure access.

2.2 Product Functionality

At a high level, the AssignHub system provides the following major functions:

- Secure role-based user authentication for students, faculty, and administrators
- Creation, publication, and management of lab assignments by faculty
- Online assignment submission and revision by students
- Automatic recording of submission timestamps and status
- Evaluation of assignments with marks and feedback
- Download of evaluated or corrected assignment files
- Tracking of assignment submission and evaluation status
- Notifications for assignment creation, deadlines, and evaluation results
- Academic structure management including courses, batches, and faculty assignments
- Administrative dashboards and basic reporting features

2.3 Design and Implementation Constraints

The design and implementation of AssignHub are subject to the following constraints:

- The system must be implemented as a web-based application using a client–server architecture
- Role-based access control must be enforced for all system functionalities
- The system must follow the COMET method for software design
- Unified Modeling Language (UML) must be used for system modeling and documentation

- All academic, user, and submission data must be stored securely in a centralized database
- Standard programming, documentation, and security conventions must be followed to ensure maintainability

References:
COMET Methodology – Concurrent Object Modeling and Architectural Design
UML – Unified Modeling Language (OMG Standard)

2.4 Assumption and Dependencies

The following assumptions and dependencies may affect the design and implementation of the AssignHub system:

- Students, faculty members, and administrators have access to devices with modern web browsers
- Stable internet connectivity is available for users accessing the system
- Users are provided with valid institutional credentials for system access
- Assignment files uploaded by students conform to defined file size and format constraints
- The system depends on a secure server and database environment for data storage and retrieval

Any changes to these assumptions may impact system functionality, performance, or design decisions.

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

- Students shall interact with the system through a web-based user interface to view assignments, submit lab work, and track submission and evaluation status.
- Faculty members shall access the system through a web-based dashboard to create assignments, monitor submissions, and evaluate student work.
- Administrators shall use an administrative interface to manage users, courses, batches, and system configurations.
- The user interface shall provide clear navigation menus, forms, and dashboards appropriate to each user role.

3.1.2 Hardware Interfaces

The AssignHub system interacts with the following hardware components:

- **Client Devices (Desktop/Laptop/Mobile):** Used by students, faculty, and administrators to access the system via a web browser.
- **Server System**: Hosts the AssignHub application, processes user requests, and manages data storage.

3.1.3 Software Interfaces

- **Web Browser**: Used by all users to access the AssignHub system.
- **Backend Server Application**: Processes authentication, assignment handling, evaluation, and notifications.
- **Database Management System**: Stores user information, assignments, submissions, evaluations, and academic data securely.

3.2 Functional Requirements

This section describes the detailed functional requirements of the AssignHub system. Each requirement is uniquely identified and structured to ensure traceability, clarity, and completeness.

R1: User Authentication & Access Control

Description: The system shall allow registered users to securely authenticate and access the system based on assigned roles.

R1.1 Login

Description: The function allows a registered user to authenticate and access the system.

- **State:** User not authenticated
- **Input:** Username/Email, Password
- **Output:** Role-based dashboard or error message
- **Processing:** Validate credentials and identify user role (Student/ Faculty/ Administrator).

R1.1.1 Validate Credentials

- **Processing:** Match credentials with stored records.

R1.1.2 Identify User Role

- **Processing:** Redirect to Student / Faculty / Admin dashboard.

R1.2 Logout

- **State:** User is authenticated
- **Input:** Logout request
- **Output:** User session terminated
- **Processing:** Invalidate the active session.

R1.3 Password Management

- **Input:** Old password, New password
- **Output:** Password update confirmation
- **Processing:** Validate old password and securely update new password.

R2: User Profile Management

Description: The system shall allow authenticated users to view and update their own permitted profile details.

R2.1 View Profile

- **State:** User is authenticated
- **Input:** Profile view request
- **Output:** Display user profile details
- **Processing:** Retrieve profile information of the logged-in user.

R2.2 Update Profile

- **State:** User is authenticated
- **Input:** Editable profile details (Name, email, password, contact details)
- **Output:** Profile update confirmation
- **Processing:** Validate input data and Update user record

⚠ **Note:** *Role, course, account status and batch cannot be modified by the user.*

R3: User Account Administration

Description: The system shall allow administrators to manage all user accounts.

R3.1 Create User

Description: The function allows the administrator to create a new user account.

- **Input:** User details, role
- **Output:** Account creation confirmation
- **Processing:** Validate user details and store account information.

R3.2 Update User

Description: The function allows the administrator to modify user details.

- **Input:** Role, course, batch changes
- **Output:** Update confirmation
- **Processing:** Validate inputs and update user records

R3.3 Deactivate User

Description: The function allows the administrator to disable a user account.

- **State:** User account exists
- **Processing:** Disable user access without deleting data

R4: Academic Structure Management

Description: The system shall manage academic entities as courses, batches, and faculty-course-batch association required for assignment handling.

R4.1 Course Management

Description: The system shall allow administrators to create and update course information used for academic activities.

R4.1.1 Create Course

Description: This function allows the administrator to add a new course to the system.

- **State:** Administrator is authenticated
- **Input:** Course code, course name, semester, department
- **Output:** Course creation confirmation
- **Processing:** Validate course details and store them in the database.

R4.1.2 Update Course

Description: This function allows the administrator to modify existing course details.

- **State:** Course exists in the system
- **Input:** Modified course details
- **Output:** Course update confirmation
- **Processing:** Validate inputs and update course records.

R4.2 Batch Management

Description: The system shall allow administrators to manage student batches for academic organization.

R4.2.1 Create Batch

Description: This function allows the administrator to add a new academic batch.

- **State:** Administrator is authenticated
- **Input:** Batch name, academic year
- **Output:** Batch creation confirmation
- **Processing:** Store batch information in the system**.**

R4.2.2 Assign Students to Batch

Description: This function allows the administrator to assign student to an academic batch.

- **State:** Batch exists
- **Input:** Student list, batch identifier
- **Output:** Student-batch mapping confirmation
- **Processing:** Associate students with the selected batch.

R4.3 Faculty-Course-Batch Mapping

Description: The system shall allow administrators to assign a faculty member to a course and batch.

- **State:** Course and batch exist
- **Input:** Faculty ID, course, batch
- **Output:** Faculty assignment confirmation
- **Processing:** Store faculty-course-batch mapping details

R5: Assignment Management

Description: The system shall allow faculty to create, publish, and manage assignments for academic courses.

R5.1 Create Assignment

Description: The function allows faculty members to create a new assignment.

- **State:** Faculty is authenticated
- **Input:** Assignment title, description, start date, due date, maximum marks
- **Output:** Assignment creation confirmation
- **Processing:** Validate assignment details and store them in the system.

R5.2 Publish / Unpublish Assignment

Description: The function allows the faculty members to control the visibility of an assignment.

- **State:** Assignment exists
- **Input:** Visibility selection (Publish / Unpublish)
- **Output:** Assignment visibility update confirmation
- **Processing:** Update assignment availability for students.

R5.3 Assignment Browsing

Description: The function allows users to view assignments relevant to their enrolled courses.

- **State:** User is authenticated
- **Input:** Course selection
- **Output:** List of assignments for enrolled courses
- **Processing:** Retrieve assignments based on user enrolment and role.

R6: Assignment Submission Handling

Description: The system shall allow students to submit, revise, and track assignment submissions.

R6.1 Submit Assignment

Description: The function allows students to submit an assignment.

- **State:** Assignment is published and within deadline
- **Input:** Assignment file
- **Output:** Submission confirmation with timestamp
- **Processing:** Validate the submitted file and store the submission details.

R6.1.1 Validate File

Description: This function validates the submitted assignment file.

- **Processing:** Check file size and permitted file format.

R6.1.2 Store Submission

Description: This function stores the assignment submission.

- **Processing:** Save the assignment file and record the submission timestamp**.**

R6.2 Revise Submission

Description: The function allows a student to revise an existing submission before the deadline.

- **State:** Submission exists and deadline not passed
- **Input:** Updated assignment file
- **Output:** Updated submission confirmation
- **Processing:** Replace the previous submission with the new file.

R6.3 Track Submission Status

Description: The function allows a student to view the current status of an assignment submission.

- **State:** User is authenticated
- **Output:** Submission Status
- **Status:** Not Submitted / Submitted / Late / Evaluated
- **Processing:** Retrieve and display current submission status.

R7: Evaluation & Feedback Management

Description: The system shall allow Faculty to evaluate student submissions and provide feedback.

R7.1 Evaluate Assignment

Description: The function allows faculty members to evaluate submitted assignments.

- **State:** Student submission exists
- **Input:** Marks, remarks, corrected file
- **Output:** Evaluation saved confirmation
- **Processing:** Store marks, feedback, and corrected file for the submission**.**

R7.2 View Evaluation

Description: The function allows student to view evaluation results.

- **State:** Submission has been evaluated
- **Output:** Marks and feedback
- **Processing:** Retrieve and display evaluation details for the student**.**

R8: Deadline & Policy Control

Description: The system shall manage assignment deadlines and enforce late submission policies.

R8.1 Extend Deadline

Description: The function allows faculty to extend assignment deadlines for specific student or batches.

- **State:** Assignment exists
- **Input:** New deadline, selected student or batch
- **Output:** Deadline update confirmation
- **Processing:** Update assignment deadline based on specific criteria.

R8.2 Late Submission Handling

Description: The function enforces late submission rules for assignment.

- **State:** Assignment submission attempted
- **Processing:** Compare submission time with the deadline and apply late submission policies**.**

R9: Notification & Dashboard

Description: The system shall notify users of important academic events and present summarized academic information.

R9.1 Notifications

Description: This function generates notifications for users regarding academic activities.

- **Triggers:** New assignment creation, approaching deadline, evaluation release
- **Output:** System notification
- **Processing:** Generate and deliver notification to relevant users**.**

R9.2 Dashboard View

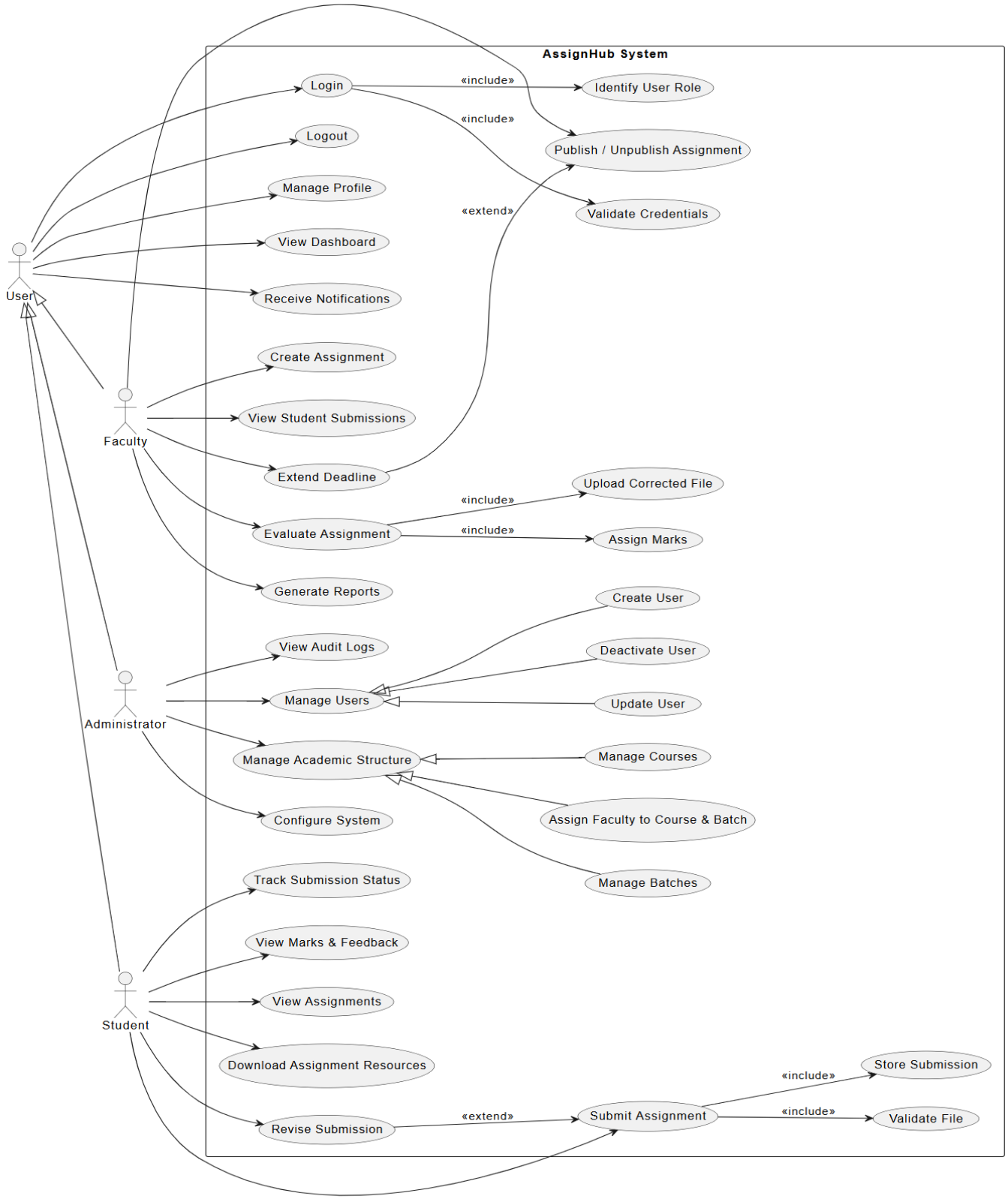
Description: The system shall display a summary of academic activities to the user.

- **State:** User is authenticated
- **Output:** Pending assignments, submission status, evaluation summary
- **Processing:** Aggregate and display user-specific academic data**.**

3.3 Use Case Model

The use case model illustrates the functional interactions between the AssignHub system and its users. The system involves three primary actors: **Student**, **Faculty**, and **Administrator**. Each actor interacts with the AssignHub system to perform role-specific operations such as assignment submission, evaluation, and academic administration.

The use case diagram encapsulates the entire AssignHub system boundary and highlights all major use cases including authentication, assignment management, submission handling, evaluation, and administrative control.


AssignHub System
Login
Logout
Manage Profile
View Dashboard
Receive Notifications
Identify User Role
Publish / Unpublish Assignment
Validate Credentials
«include»
«include»
«extend»
User
Faculty
Administrator
Student
Create Assignment
View Student Submissions
Extend Deadline
Evaluate Assignment
Upload Corrected File
Assign Marks
«include»
«include»
Generate Reports
View Audit Logs
Manage Users
Create User
Deactivate User
Update User
Manage Academic Structure
Manage Courses
Assign Faculty to Course & Batch
Manage Batches
Configure System
Track Submission Status
View Marks & Feedback
View Assignments
Download Assignment Resources
Revise Submission
«extend»
Submit Assignment
«include»
Store Submission
«include»
Validate File

3.3.1 Use Case #1 – Submit Assignment (UC-1)

Author: Solanki Vimal
Purpose: This use case allows a student to upload and submit an assignment file for a published assignment before the deadline.

Requirements Traceability:
R6.1, R6.1.1, R6.1.2, R6.2, R6.3

Priority: High

Preconditions:
- Student is authenticated
- Assignment is published
- Submission deadline has not passed

Postconditions:
- Assignment file is stored successfully
- Submission timestamp is recorded
- Submission status is updated

Actors:
- Student

Extends:
- Revise Submission (UC-1a)

Flow of Events:

1. Basic Flow

1. Student selects a published assignment
2. Student uploads assignment file
3. System validates file format and size
4. System stores submission and records timestamp
5. System displays submission confirmation

2. Alternative Flow
- Student replaces an existing submission before the deadline

3. Exceptions

- Invalid file format
- File size exceeds limit
- Deadline expired

Includes:

- Validate File
- Store Submission

Notes/Issues:
Late submissions are handled according to faculty-defined policies.

3.3.2 Use Case #2 – Evaluate Assignment (UC-2)

Author: Hanuj Trivedi
Purpose: This use case enables faculty members to evaluate student submissions, assign marks, and provide feedback.

Requirements Traceability:
R7.1, R7.2

Priority: High

Preconditions:
- Faculty is authenticated
- Student submission exists

Postconditions:
- Marks and feedback are stored
- Evaluation status is updated

Actors:
- Faculty

Extends: None

Flow of Events:

1. Basic Flow
1. Faculty selects assignment
2. Faculty opens student submission
3. Faculty enters marks and feedback
4. Faculty uploads corrected file
5. System saves evaluation

2. Alternative Flow
- Faculty updates evaluation before final submission

3. Exceptions
- Submission file unavailable
- Invalid marks entry

Includes:

- Assign Marks
- Upload Corrected File

Notes/Issues:
Evaluation visibility is restricted until released.

3.3.3 Use Case #3 – Manage Academic Structure (UC-3)

Author: Solanki Vimal
Purpose: Allows administrators to manage courses, batches, and faculty-course-batch mappings.

Requirements Traceability:
R4.1, R4.2, R4.3

Priority: Medium

Preconditions:
- Administrator is authenticated

Postconditions:
- Academic structure data updated

Actors:
- Administrator

Flow of Events:

1. Admin creates/updates courses
2. Admin creates batches
3. Admin assigns faculty to courses and batches

3.3.4 Use Case #4 – Authenticate User (UC-4)

Author: Hanuj Trivedi
Purpose: Ensures secure login and role-based access for all users.

Requirements Traceability:
R1.1, R1.1.1, R1.1.2, R1.2

Priority: High

Actors:
- Student, Faculty, Administrator

Includes:

- Validate Credentials
- Identify User Role

4. Other Non-functional Requirement

4.1 Performance Requirements

The AssignHub system shall be designed to provide acceptable performance under normal academic usage conditions.

- The system shall support multiple concurrent users, including students submitting assignments and faculty evaluating submissions simultaneously.
- The average response time for loading pages such as dashboards, assignment lists, and submission status shall not exceed **3 seconds** under normal load.
- Assignment file uploads and submission confirmations shall be completed within **5 seconds** for standard file sizes.
- Notifications related to assignments, deadlines, and evaluations shall be generated and delivered in near real time after triggering events.

These performance requirements ensure smooth system operation during peak academic periods such as assignment deadlines.

4.2 Safety and Security Requirements

The AssignHub system shall ensure the security and protection of academic data and user information.

- All users shall be authenticated using secure login mechanisms.
- User passwords shall be stored in encrypted form in the database.
- Role-based access control shall be enforced to prevent unauthorized access to academic or administrative functionalities.
- Users shall only be able to access data relevant to their assigned roles.
- Secure communication protocols (such as HTTPS) shall be used for data transmission.
- Uploaded assignment files shall be validated to prevent malicious or unsupported content.
- User sessions shall be terminated automatically after a period of inactivity.

These requirements reduce the risk of data breaches, unauthorized access, and system misuse.

4.3 Software Quality Attributes

The AssignHub system shall satisfy the following software quality attributes:

4.3.1 Reliability

- The system shall reliably store assignment submissions, evaluations, and academic records without data loss.
- Regular backups shall be maintained to ensure recovery in case of system failure.

4.3.2 Usability

- The user interface shall be simple, consistent, and easy to navigate for students, faculty, and administrators.
- Dashboards shall present clear summaries of academic activities and statuses.

4.3.3 Maintainability

- The system shall follow a modular design to allow easy updates and enhancements.
- Clear documentation and coding standards shall be followed to simplify maintenance

5. Other Requirements

This section describes additional requirements not covered elsewhere in the SRS.

- The system shall maintain audit logs for critical administrative actions such as user creation, role updates, and account deactivation.
- Academic data, including assignments and submissions, shall be archived after the completion of each academic term.
- The database shall support structured storage of users, courses, assignments, submissions, evaluations, and notifications.
- The system shall comply with institutional policies related to data privacy and academic record management.
- The system design shall allow future extensions such as plagiarism detection or integration with other academic systems.

Appendix A – Data Dictionary

The data dictionary defines the key data elements, system states, inputs, and outputs used in the AssignHub system. It provides a clear reference for understanding how data is stored, processed, and exchanged across different system functionalities.

A.1 Data Elements

Data Item	Description	Used In / Related Requirements
User_ID	Unique identifier for each system user	R1, R2, R3
User_Role	Role assigned to the user (Student / Faculty / Admin)	R1.1.2
Username	Login identifier for authentication	R1.1
Password	Encrypted credential for user authentication	R1.1, R1.3
Course_ID	Unique identifier for a course	R4.1
Batch_ID	Unique identifier for an academic batch	R4.2
Assignment_ID	Unique identifier for an assignment	R5
Assignment_Title	Title of the assignment	R5.1
Assignment_Description	Details and instructions for assignment	R5.1
Due_Date	Submission deadline for assignment	R5.1, R8
Submission_ID	Unique identifier for assignment submission	R6
Submission_File	Uploaded assignment file	R6.1
Submission_Time	Timestamp of assignment submission	R6.1
Submission_Status	Current state of submission	R6.3
Marks	Marks assigned by faculty	R7.1
Feedback	Remarks provided by faculty	R7.1
Notification	System-generated alert message	R9.1

A.2 State Variables

State Variable	Possible States	Description
User_State	Authenticated / Not Authenticated	Represents login status
Assignment_State	Published / Unpublished	Controls assignment visibility
Submission_Status	Not Submitted / Submitted / Late / Evaluated	Tracks submission lifecycle
Evaluation_State	Pending / Completed	Tracks evaluation progress

A.3 Inputs and Outputs

Input / Output	Description	Related Requirements
Login Credentials	Username and password provided by user	R1.1
Assignment File	File uploaded by student	R6.1
Marks & Feedback	Evaluation input by faculty	R7.1
Notifications	Alerts generated by system	R9.1
Dashboard Summary	Aggregated academic data	R9.2

Appendix B – Group Log

This section documents the major group activities carried out during the preparation of the AssignHub project and its Software Requirements Specification (SRS), starting from project selection to final documentation.

Date	Activity Description
01/12/2025	Project selection and problem identification carried out during Lab 1. Topic **AssignHub – Centralized Lab Assignment Submission and Evaluation System** was finalized.
08/12/2025	Identification of system users (Student, Faculty, Administrator) and discussion of user responsibilities during Lab 2.
15/12/2025	Basic system prototype and initial workflow discussion conducted during Lab 3.
22/12/2025	Identification and documentation of detailed functional requirements as part of Lab 4.
05/01/2026	SRS document development initiated during Lab 5 based on collected requirements.
12/01/2026	Completion of **Draft 1.0** of the SRS, covering system overview, scope, assumptions, and functional requirements.
19/01/2026	Development of use case model and detailed use case descriptions during Lab 6; SRS updated to **Draft 2.0**.
09/02/2026	Addition of non-functional requirements, other requirements, appendices, and final review; **Draft 3.0** completed.